BUT2 INFORMATIQUE · RESSOURCE R3.05

Programmation système

CM 02 — Les fichiers

```
bash — R3.05
$ ls -li essai.c
1835 -rw-r--r-- 1 cruz info 1024 essai.c
$ stat -c "%i %s octets" essai.c
1835 1024 octets
```

Christophe Cruz · christophe.cruz@ube.fr · 2026-2027

IUT Dijon-Auxerre · Université Bourgogne Europe

Au programme

01

Le fichier

Pourquoi des fichiers ? Une abstraction : la suite d'octets.

02

Le SGF vu par l'utilisateur

Noms, types, droits, répertoires, chemins, arborescence Linux.

03

Les appels système

Descripteurs, open, read, write, lseek, close, opendir.

04

Implémentation

Partitions, blocs, allocation, i-nœuds, répertoires.

Prérequis : CM 01a (pointeurs, tableaux) et CM 01b (stdin, stdout, stderr = descripteurs 0, 1, 2).

01

Le fichier

Une abstraction pour stocker les données durablement

Pourquoi des fichiers ?

01

Grande quantité

Enregistrer bien plus d'informations que la mémoire vive ne peut en contenir.

02

Persistance

Conserver les données après l'arrêt du processus… et de la machine.

03

Partage

Permettre à plusieurs processus d'accéder aux mêmes données, en même temps.

La solution : **le fichier, une suite d'octets**

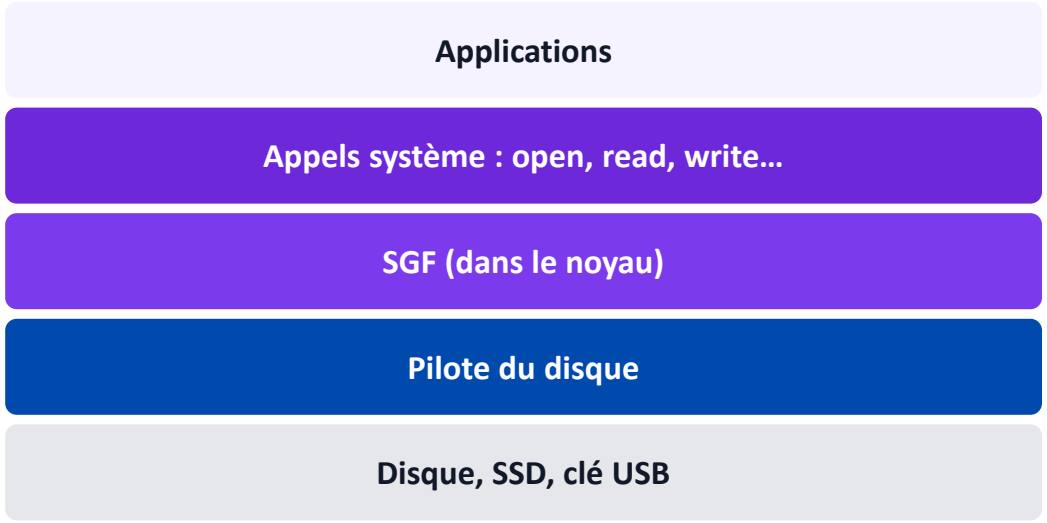
Le système structure les données sur le support physique et ne montre aux programmes qu'une suite d'octets numérotés. Le sens de ces octets (texte, image, exécutable) dépend du programme qui les lit.



contenu de bonjour.txt (en hexadécimal)

Le système de gestion de fichiers (SGF)

- Les fichiers sont gérés par le SGF, une partie du noyau
- Il fait le lien entre les noms de fichiers et les blocs du disque
- Il existe un SGF adapté à chaque besoin



ext4

Linux

NTFS

Windows

APFS

macOS

exFAT / FAT32

clés USB, cartes SD

ISO 9660

CD-ROM

NFS

fichiers en réseau

02

Le SGF vu par l'utilisateur

Noms, types, droits, répertoires et chemins

Le nommage des fichiers

Un fichier a un nom et souvent une extension. Les règles dépendent du SGF.

MS-DOS (FAT)

8 caractères + extension facultative de 3 (format 8.3).

Insensible à la casse : jean.cpp, Jean.cpp et JEAN.CPP sont le même fichier.

UNIX / Linux

Jusqu'à 255 octets, tous les caractères sauf / et \0.

Sensible à la casse : ToTo ≠ toto. L'extension n'est qu'une convention : prog.c.tar.gz.

Windows (NTFS)

Noms en Unicode, jusqu'à 255 caractères.

Conserve la casse à l'affichage, mais n'en tient pas compte pour comparer.

sous Linux : deux fichiers distincts

```
$ touch toto ToTo
$ ls
ToTo  toto
```

Attention aux échanges : copier toto et ToTo sur une clé USB en FAT32 ou sous Windows provoque un conflit de noms.

Les types de fichiers

- **Fichier ordinaire**
Les données des utilisateurs : texte, image, programme…

d **Répertoire**
Un fichier qui conserve la structure du SGF (noms → fichiers).

c **Spécial caractère**
Périphérique lu octet par octet : terminal, port série (/dev/tty).

b **Spécial bloc**
Périphérique lu par blocs : disque (/dev/sda).

l **Lien symbolique**
Un raccourci vers un autre chemin.

p **Tube nommé**
Un canal de communication (CM 03b).

le 1er caractère = le type

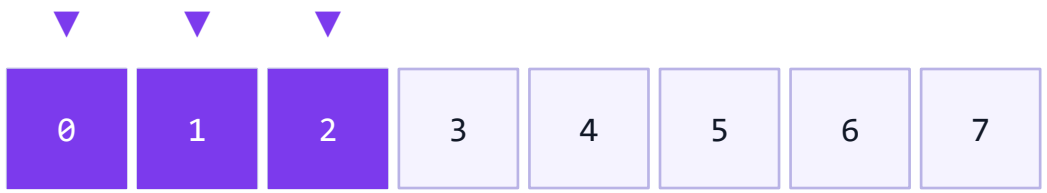
```
$ ls -ld /etc /dev/tty /dev/sda essai.c
drwxr-xr-x  1 root root    4096 /etc
crw-rw-rw-  1 root tty   5,   0 /dev/tty
brw-rw----  1 root disk  8,   0 /dev/sda
-rw-r--r--  1 cruz info    1024 essai.c
```

Sous UNIX, tout est fichier !

Accès séquentiel ou aléatoire

Accès séquentiel

Les octets (ou enregistrements) sont lus dans l'ordre, du début à la fin. C'est le mode des bandes magnétiques.



Bande magnétique LTO-9
Jusqu'à 18 To par cartouche (45 To compressés). Encore utilisée pour les sauvegardes et l'archivage : peu chère et durable, mais lente pour accéder à une donnée précise.

Accès aléatoire (direct)

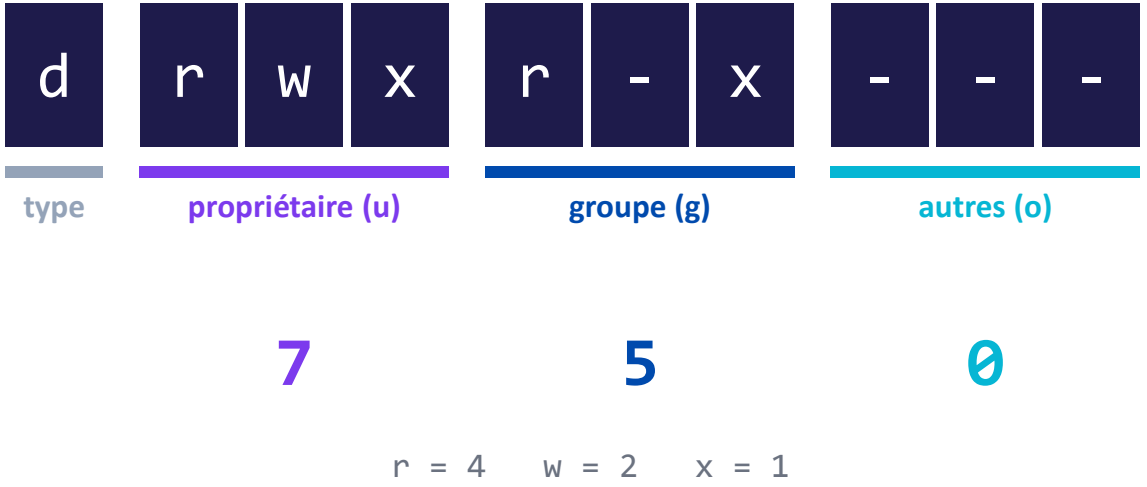
On lit dans n'importe quel ordre, en deux étapes : se positionner (seek), puis lire (read). C'est le mode des disques et des SSD.



En C : lseek() déplace la position courante, read() et write() lisent et écrivent à partir de cette position (partie 03).

Attributs et droits d'accès

- Les attributs varient selon le système : dates (création, modification, accès), taille, propriétaire, fichier caché, archive, verrouillage…
- Sous UNIX : 3 catégories d'utilisateurs : u (user, propriétaire), g (group), o (others)
- 3 droits : r (lire), w (modifier), x (exécuter un fichier, ou traverser un répertoire)

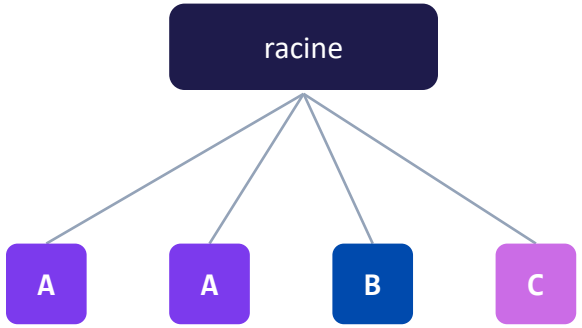


Répertoire sans x : on peut lister les noms (r), mais pas entrer dedans ni ouvrir ses fichiers.

```
$ ls -ld /home/toto
drwxr-x--- 2 toto etudiant 4096 /home/toto
$ chmod 750 /home/toto     # même chose en octal
```

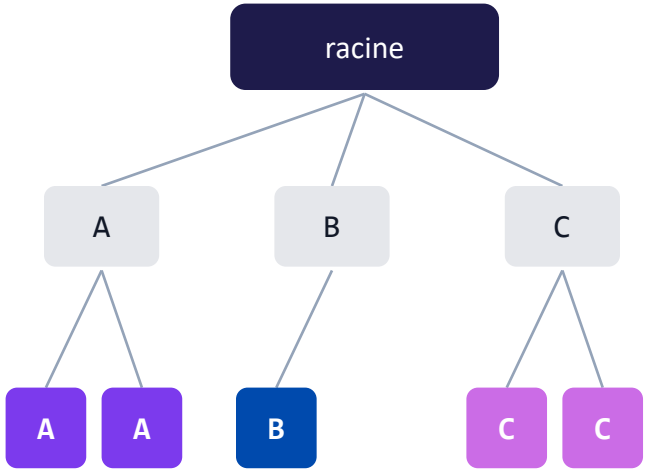
Les répertoires : de 1 à n niveaux

Un seul niveau



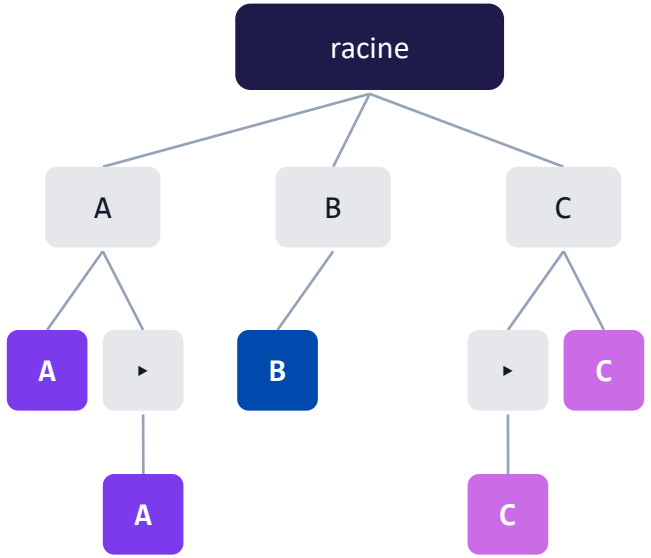
Ex. CDC 6600 (1964), petits systèmes embarqués. Conflits de noms entre utilisateurs.

Deux niveaux



Un répertoire privé par utilisateur : plus de conflits.

Hiérarchique



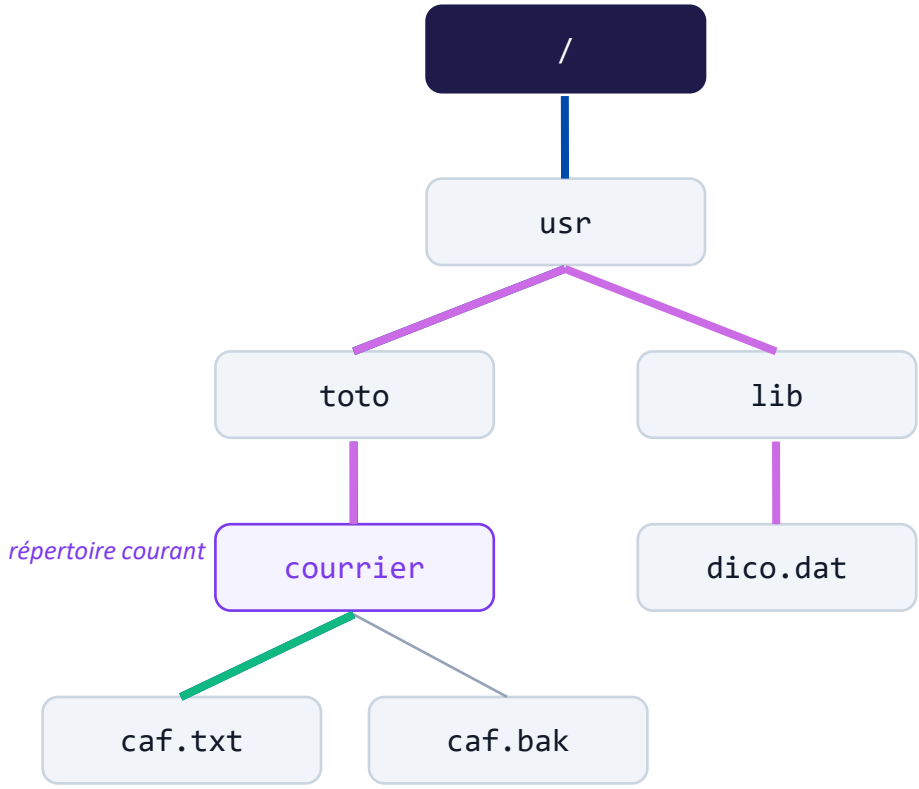
Chaque utilisateur crée l'arbre qu'il veut (sous-répertoires).

A, B, C : fichiers de trois propriétaires

Chemins absolus et relatifs

Système	Séparateur	Exemple
UNIX / Linux	/	/usr/toto/courrier/
Windows	\	C:\Users\toto\Documents\
MULTICS	>	>usr>toto>courrier

- Chemin absolu : commence par le séparateur, il part de la racine
- Chemin relatif : part du répertoire de travail (répertoire courant)
- « . » = le répertoire courant, « .. » = le répertoire parent




```
$ cd /usr/toto/courrier
$ cp /usr/toto/courrier/caf.txt /usr/toto/courrier/caf.bak
$ cp ./caf.txt ./caf.bak          # la même chose
$ cp ../lib/dico.dat .
```

absolu : on part de /

relatif : on part de courrier

.. remonte vers toto, puis on redescend

L'arborescence Linux

/bin

Commandes de base : ls, cat, cp, mv…

/sbin

Commandes d'administration : fsck, mkfs…

/dev

Fichiers spéciaux des périphériques : /dev/sda1 (1re partition du 1er disque), /dev/tty, /dev/null

/etc

Configuration : /etc/passwd (comptes), /etc/shadow (mots de passe chiffrés), /etc/fstab (disques à monter)

/home

Les répertoires des utilisateurs

/lib

Bibliothèques partagées (libc…)

/tmp

Fichiers temporaires, effacés au redémarrage

/usr

Programmes et données des utilisateurs : /usr/bin, /usr/lib, /usr/include (stdio.h !)

/var

Données qui varient : journaux (/var/log), files d'impression, courriers, caches

Les liens symboliques

Un lien symbolique est un petit fichier qui contient le chemin d'un autre fichier : c'est un raccourci.

```
$ ln -s /home/olivier/lettre.txt ./imprimer/lettre.txt
$ ls -l ./imprimer/lettre.txt
lrwxrwxrwx 1 olivier etudiant 24 janv. 14:16 ./imprimer/lettre.txt -> /home/olivier/lettre.txt
```



contenu : « /home/olivier/lettre.txt »

Si la cible est supprimée, le lien devient « cassé ». À ne pas confondre avec le lien physique (ln sans -s), qui donne un 2e nom au même fichier (partie 04).

03

Les appels système

open, read, write, lseek, close — le plus bas niveau

La table des descripteurs

- Chaque processus possède une table des descripteurs de fichiers
- Un descripteur est un simple entier : l'indice dans cette table
- Tous les fichiers se manipulent de la même façon : open, read, write, close
- open choisit toujours le plus petit numéro libre
- Le nombre de descripteurs est limité (ulimit -n : souvent 1024)

desc.	fichier	i-nœud
0	/dev/stdin (clavier)	234
1	/dev/stdout (écran)	254
2	/dev/stderr (écran)	112
3	monfichier.c	324
4	monfichier.o	554
5		

```c
int fd = open("monfichier.c", O_RDONLY);   // fd vaut 3
```

Ouvrir un fichier : open

```c
#include <fcntl.h>

int open(const char *chemin, int drapeaux);
int open(const char *chemin, int drapeaux, mode_t mode);
```

- chemin : chemin relatif ou absolu du fichier
- drapeaux : mode d'accès, combiné avec | (ou bit à bit)
- mode : droits du fichier s'il doit être créé
- Renvoie le descripteur, ou -1 en cas d'erreur

Drapeau	Effet
O_RDONLY	lecture seule
O_WRONLY	écriture seule
O_RDWR	lecture et écriture
O_CREAT	créer le fichier s'il n'existe pas
O_TRUNC	vider le fichier à l'ouverture
O_APPEND	écrire toujours à la fin
O_EXCL	avec O_CREAT : erreur si le fichier existe

Un seul des trois premiers, combiné avec les autres.

```c
int fd = open("notes.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
```

Avec O_CREAT, le 3e argument est obligatoire : sinon les droits du fichier créé sont aléatoires.

Droits effectifs et umask

Les droits réellement donnés au fichier créé sont ceux demandés, privés des bits du masque du processus (umask).

```
open("monfichier.c", O_RDWR | O_CREAT, 0666);
```

mode demandé	0666	r	w	-	r	w	-	r	w	-
umask	0022	-	-	-	-	w	-	-	w	-
droits obtenus	0644	r	w	-	r	-	-	r	-	-

```
$ umask
0022
$ touch nouveau
$ ls -l nouveau
-rw-r--r-- 1 cruz info 0 nouveau
```

calcul : mode & ~umask

Traiter les erreurs : errno et perror

- Quand un appel système échoue, il renvoie -1 et fixe la variable globale errno (<errno.h>)
- perror(s) affiche s, puis le message correspondant à errno
- strerror(errno) renvoie ce message sous forme de chaîne

errno	Signification
ENOENT	fichier ou répertoire inexistant
EACCES	permission refusée
EEXIST	le fichier existe déjà (O_EXCL)
EMFILE	trop de fichiers ouverts

erreur.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>

int fd = open("./monfichier", O_WRONLY);
if (fd == -1) {
    perror("open monfichier");
    exit(EXIT_FAILURE);
}
```

```
$ ./erreur
open monfichier: No such file or directory
```

Règle : chaque appel système, son test d'erreur.

Lire : read

```c
#include <unistd.h>

ssize_t read(int fd, void *buf, size_t nb);
```

- fd : descripteur d'un fichier ouvert en lecture
- buf : adresse de la zone mémoire qui reçoit les octets lus
- nb : nombre d'octets que l'on veut lire
- Les lectures sont séquentielles : chaque read reprend là où le précédent s'est arrêté

Valeur renvoyée

`-1`	erreur (voir errno)
`0`	fin du fichier atteinte
`n > 0`	nombre d'octets lus, parfois moins que nb

`ssize_t` est un entier signé (pour pouvoir renvoyer -1), size_t un entier non signé.

read pas à pas

lecture.c

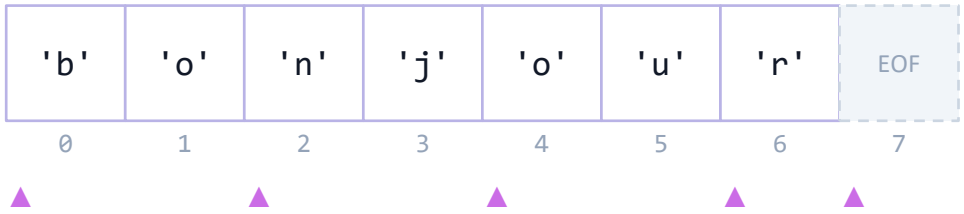
```c
char tab[10];
int fd = open("monfichier", O_RDONLY);
n = read(fd, tab, 2);
n = read(fd, tab, 2);
n = read(fd, tab, 2);
n = read(fd, tab, 2);
n = read(fd, tab, 2);
```

→ n = 2
→ n = 2
→ n = 2
→ n = 1
→ n = 0 : fin

À observer

Chaque read écrit à partir de tab[0] : il écrase la lecture précédente.
Le 4e read ne trouve plus qu'un octet : tab vaut "ru", le u vient du read d'avant !
Le 5e renvoie 0 : c'est ainsi qu'on détecte la fin du fichier.

monfichier (sur le disque)



▲ = *position courante dans le fichier*

tab (en mémoire)



Écrire : write

```
ssize_t write(int fd, const void *buf, size_t nb);
```

- fd : descripteur ouvert en écriture
- buf : adresse des octets à écrire
- nb : nombre d'octets à écrire
- Renvoie -1 en cas d'erreur, sinon le nombre d'octets écrits, qui peut être inférieur à nb (disque plein, tube, signal…)

copie.c

```c
/* copie src -> dst, comme la commande cp */
char buf[4096];
ssize_t n;

while ((n = read(src, buf, sizeof buf)) > 0) {
    if (write(dst, buf, n) != n) {
        perror("write");
        exit(EXIT_FAILURE);
    }
}
if (n == -1)
    perror("read");
```

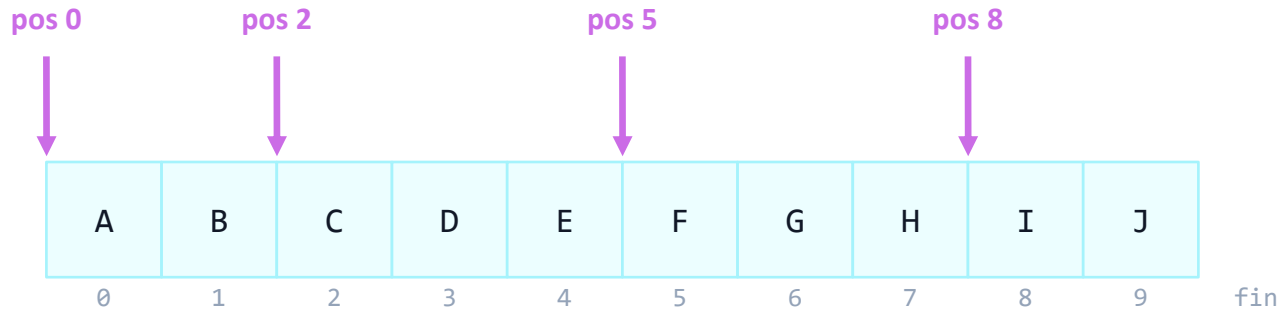
On n'écrit que n octets : le dernier read lit souvent moins que 4096.

Se positionner : lseek

```
off_t lseek(int fd, off_t decalage, int origine);
```

origine	Nouvelle position
SEEK_SET	décalage depuis le début du fichier
SEEK_CUR	position courante + décalage
SEEK_END	taille du fichier + décalage

Le décalage peut être négatif. lseek renvoie la nouvelle position, ou -1. Astuce : lseek(fd, 0, SEEK_END) donne la taille du fichier.




```
lseek(fd, 2, SEEK_SET);      → 2
lseek(fd, 3, SEEK_CUR);      → 5
lseek(fd, -2, SEEK_END);     → 8
```

Avant : lseek(fd, 0, 0), avec 0, 1, 2 en dur. Préférez SEEK_SET, SEEK_CUR, SEEK_END : c'est lisible et portable.

Fermer : close

```c
int close(int fd);        // renvoie 0, ou -1 en cas d'erreur
```

01

Libère le numéro

Le descripteur redevient libre : le prochain open pourra le réutiliser.

02

Fermeture automatique

Les fichiers sont fermés par le système quand le processus se termine…

03

… mais fermez quand même

Le nombre de descripteurs est limité, et un tube non fermé empêche de détecter la fin (CM 03b).

```c
int fd = open("notes.txt", O_RDONLY);
if (fd == -1) { perror("open"); exit(EXIT_FAILURE); }
/* ... read ... */
if (close(fd) == -1)
    perror("close");
```

Lire et écrire des données binaires

read et write manipulent des octets : on peut donc écrire directement la mémoire d'une variable, en respectant quelques règles.

binaire.c

```c
struct point { float x; float y; };
struct point a = {1.5, 2.5}, b;

write(fd, &a, sizeof a);    // 8 octets
/* ... */
read(fd, &b, sizeof b);
```

```
$ xxd point.bin
00000000: 0000 c03f 0000 2040    ...?.. @
```

`1.5 = 0x3FC00000` rangé à l'envers (00 00 C0 3F) : processeur little-endian

Taille

Utilisez sizeof : jamais de taille écrite en dur.

Format

Le fichier n'est pas lisible dans un éditeur de texte.

Portabilité

Boutisme et alignement varient : ne relisez que sur le même type de machine.

Créer et supprimer des répertoires

```c
#include <sys/stat.h>
#include <unistd.h>

int mkdir(const char *chemin, mode_t mode);   // crée un répertoire vide
int rmdir(const char *chemin);                // supprime un répertoire VIDE
int unlink(const char *chemin);               // supprime un fichier
```

repertoires.c

```c
if (mkdir("sauvegardes", 0755) == -1)
    perror("mkdir");          // EEXIST si le répertoire existe déjà

if (rmdir("sauvegardes") == -1)
    perror("rmdir");          // ENOTEMPTY s'il contient des fichiers
```

Comme pour open, le mode passe par le umask : 0755 & ~022 = 0755 (rwxr-xr-x).

Lire un répertoire : opendir / readdir

mini_ls.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <dirent.h>

int main(int argc, char *argv[])
{
    const char *chemin = argc > 1 ? argv[1] : ".";
    DIR *d = opendir(chemin);
    if (d == NULL) { perror("opendir"); exit(EXIT_FAILURE); }

    struct dirent *e;
    while ((e = readdir(d)) != NULL)
        printf("%8lu  %s\n", (unsigned long) e->d_ino, e->d_name);

    closedir(d);
    return 0;
}
```

terminal

```
$ ./mini_ls
 1835201  .
 1835009  ..
 1835206  mini_ls.c
 1835210  mini_ls
 1835208  notes.txt
```

- opendir ouvre le répertoire (NULL si erreur)
- readdir renvoie l'entrée suivante, NULL à la fin
- d_name : le nom ; d_ino : le n° d'i-nœud
- Ces fonctions de la libc s'appuient sur l'appel système getdents

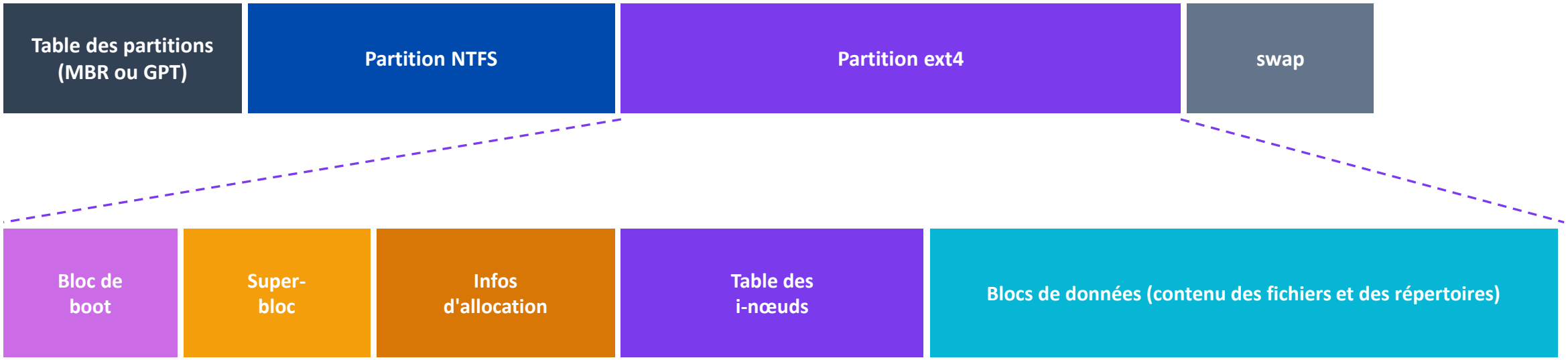
04

Implémentation du SGF

Comment le SGF range les fichiers sur le disque

Partitions du disque

Organisation d'un disque



Super-bloc : Taille de la partition, taille des blocs, nombre d'i-nœuds…

Infos d'allocation : Quels blocs et quels i-nœuds sont libres (bitmaps)

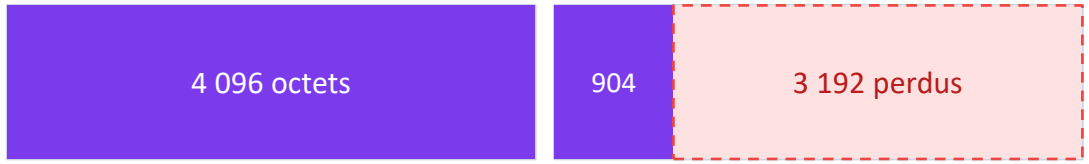
Table des i-nœuds : Une fiche par fichier : droits, taille, dates, blocs

```
$ lsblk
NAME     SIZE TYPE MOUNTPOINT
sda      512G disk
├─sda1     1G part /boot/efi
├─sda2   495G part /
└─sda3    16G part [SWAP]
```

La notion de bloc

- Le disque se lit par secteurs : 512 octets (4096 sur les disques récents)
- Le SGF regroupe les secteurs en blocs : une puissance de deux, de 1 à 64 Kio
- ext4 et NTFS utilisent par défaut des blocs de 4 Kio
- Un fichier occupe toujours un nombre entier de blocs

Fichier de 5 000 octets, blocs de 4 096 octets



Le compromis : de gros blocs accélèrent la lecture mais gaspillent de la place pour les petits fichiers. De petits blocs font l'inverse.

Retrouver les blocs d'un fichier : 3 stratégies

Allocation contiguë

Les blocs se suivent. Ex. : CD-ROM.

Liste chaînée (FAT)

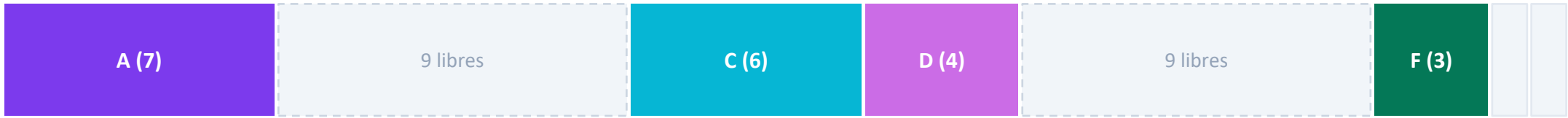
Une table indique le bloc suivant. Ex. : clés USB.

Index (i-nœuds)

Une fiche liste les blocs. Ex. : ext4.

L'allocation contiguë

Le disque : 40 blocs





← aucun trou n'est assez grand, alors que 18 blocs sont libres !

Avantage

Très simple, et lecture très rapide : il suffit de connaître le 1er bloc et la taille.

Problème 1

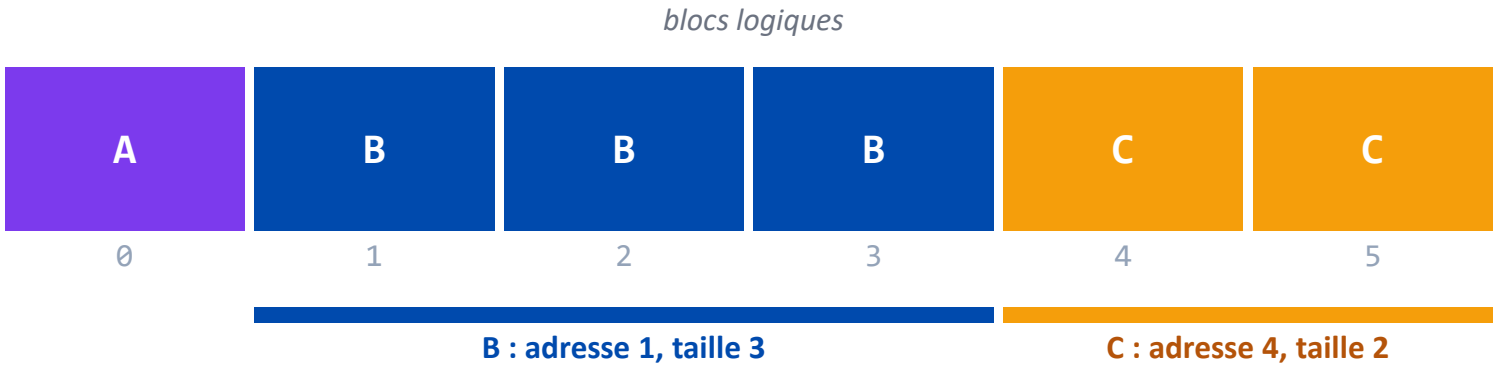
Avec le temps, le disque se fragmente en trous trop petits.

Problème 2

Il faut connaître la taille finale du fichier dès sa création. Idéal en revanche pour un CD-ROM, écrit une seule fois.

Exemple : le système de fichiers d'un CD-ROM

Sur un CD-ROM, tout est écrit en une fois : l'allocation contiguë est parfaite. Un répertoire donne simplement l'adresse et la taille de chaque fichier.



Nom	Adresse	Taille
B	1	3
C	4	2

Le fichier A est un répertoire : son contenu est ce tableau.

Norme ISO 9660
Encore utilisée pour les images .iso (installation de Linux, par exemple).

La liste chaînée : la FAT

- La FAT (File Allocation Table) a une case par bloc du disque
- Chaque case indique le bloc suivant du fichier, ou une marque de fin
- Le répertoire ne donne que le 1er bloc
- Utilisée par FAT32 et exFAT : clés USB, cartes SD

Nom	1er bloc
photo.jpg	4

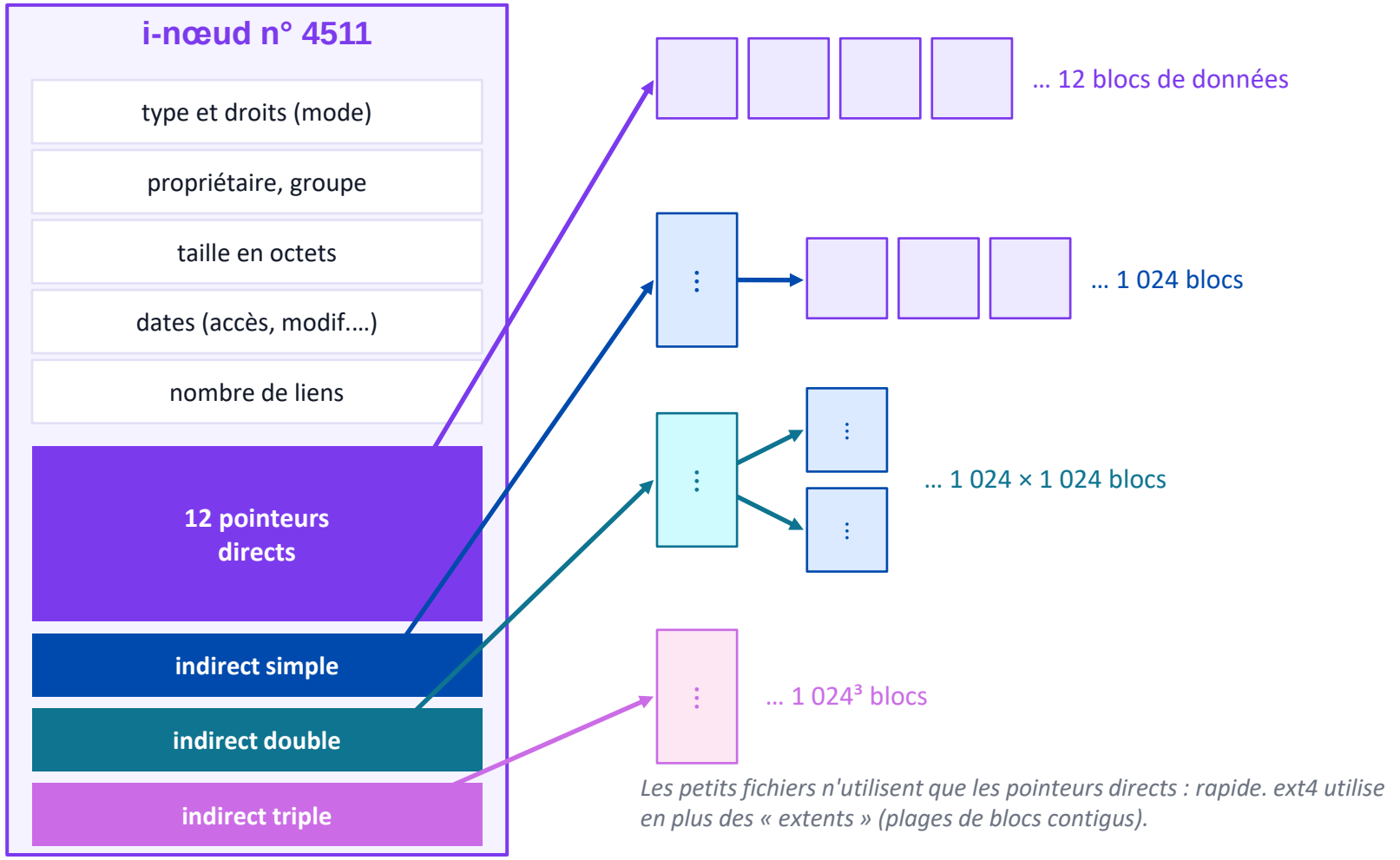
bloc	FAT : suivant
0	réservé
1	réservé
2	10
3	libre
4	7
5	autre
6	autre
7	2
8	libre
9	libre
10	fin
11	libre



blocs de photo.jpg, dans l'ordre

Limite : pour atteindre le 1000e bloc, il faut suivre 999 cases de la table.

L'i-nœud (UNIX, ext4)



Les petits fichiers n'utilisent que les pointeurs directs : rapide. ext4 utilise en plus des « extents » (plages de blocs contigus).

Taille maximale (blocs de 4 Kio)

directs
`12 × 4 Kio` — **48 Kio**

+ simple
`1 024 × 4 Kio` — **4 Mio**

+ double
`1 024² × 4 Kio` — **4 Gio**

+ triple
`1 024³ × 4 Kio` — **4 Tio**

Un répertoire est un fichier

Un répertoire est un fichier spécial dont le contenu est une liste d'associations : nom → numéro d'i-nœud. Tout le reste (droits, taille, blocs) est dans l'i-nœud.

`/ (i-nœud 2)`

nom	i-nœud
.	2
..	2
bin	127
etc	4328
home	13

`/etc (i-nœud 4328)`

nom	i-nœud
.	4328
..	2
passwd	4511
group	743

`/bin (i-nœud 127)`

nom	i-nœud
.	127
..	2
ls	145
cp	7456

« . » et « .. » sont de vraies entrées : « . » pointe sur l'i-nœud du répertoire lui-même, « .. » sur celui du parent. Pour la racine, les deux valent 2.

Que fait open("/etc/passwd") ?

1. Lire l'i-nœud 2 (la racine, connue d'avance)
2. Lire les blocs de /, chercher « etc »
3. Lire l'i-nœud 4328, puis ses blocs, chercher « passwd »
4. Lire l'i-nœud 4511 : vérifier les droits, trouver les blocs
5. Ajouter une entrée dans la table des descripteurs, renvoyer 3

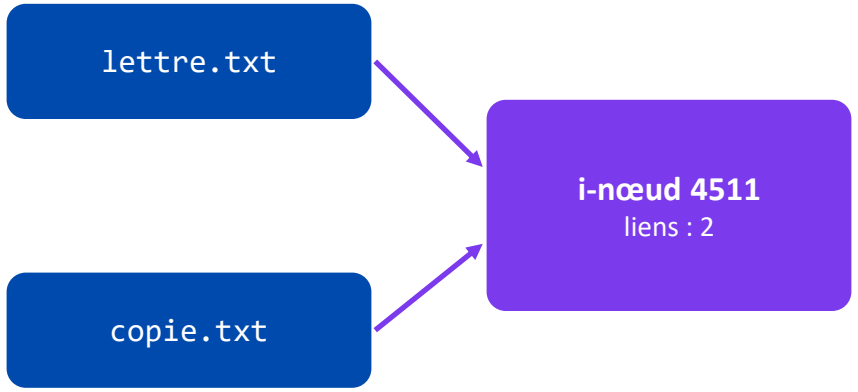


Plus le chemin est long, plus il y a de lectures. Le noyau garde donc un cache des noms récents.

Un nom n'est qu'une entrée de répertoire

Puisque le nom est dans le répertoire et le fichier dans l'i-nœud, un même fichier peut avoir plusieurs noms : ce sont les liens physiques.

```
$ ln lettre.txt copie.txt
$ ls -li lettre.txt copie.txt
4511 -rw-r--r-- 2 cruz info 120 lettre.txt
4511 -rw-r--r-- 2 cruz info 120 copie.txt
$ rm lettre.txt          # copie.txt existe toujours
$ ls -li copie.txt
4511 -rw-r--r-- 1 cruz info 120 copie.txt
```



rm ne supprime pas le fichier : il retire un nom (appel système unlink). Les blocs ne sont libérés que lorsque le nombre de liens tombe à 0 et que plus aucun processus ne l'a ouvert.

SYNTHÈSE

À retenir

01

Le fichier

Une suite d'octets nommée, gérée par le SGF du noyau. Sous UNIX, tout est fichier.

02

Droits

u / g / o × r / w / x, en octal (750). Les droits de création sont filtrés par le umask.

03

Descripteurs

open renvoie le plus petit entier libre. 0, 1 et 2 sont déjà pris.

04

read / write

Séquentiels, peuvent traiter moins que demandé. read renvoie 0 en fin de fichier.

05

Erreurs

Les appels système renvoient -1 et fixent errno : testez-les, utilisez perror.

06

i-nœuds

Le répertoire associe nom → i-nœud. L'i-nœud contient tout le reste, dont les blocs.

Ressources pédagogiques

Le manuel

```
$ man 2 open        # appels système
$ man 2 read
$ man 2 lseek
$ man 3 opendir     # bibliothèque C
$ man 7 inode       # i-nœuds
$ man 7 hier        # arborescence
```

La section 2 du manuel décrit les appels système, la section 3 les fonctions de la bibliothèque C, la section 7 les concepts.

Lectures

- A. Tanenbaum, Systèmes d'exploitation, Pearson : chapitre « Systèmes de fichiers »
- J.-M. Rifflet, La programmation sous UNIX, Ediscience
- Documentation du noyau Linux : ext4 Data Structures

kernel.org : ext4 Data Structures and Algorithms

lamae

Questions ?

PROCHAINE SÉANCE

CM 03a — Les processus

fork, exec, wait, exit · états d'un processus · signaux

Christophe Cruz · christophe.cruz@ube.fr